

■ AI/ML Internship – Day 76 Notes & Tasks

● Module 9: Multi-PDF RAG System – Building an AI Chatbot That Answers Questions from Multiple Documents

📌 Part 1: Recap of Day 75

Yesterday we learned:

- ✓ Prompt Engineering
- ✓ System Prompt
- ✓ User Prompt
- ✓ Context Prompt
- ✓ Prompt Templates
- ✓ Guardrails
- ✓ Prompt Injection

Today we will learn:

- What is Multi-PDF RAG?
 - Why Multi-PDF RAG is needed
 - How to process multiple documents
 - Merging knowledge from multiple PDFs
 - Handling duplicate information
 - Building a Multi-PDF Question Answering chatbot
-

📌 Part 2: What is Multi-PDF RAG?

Definition

A Multi-PDF RAG System is a Retrieval-Augmented Generation application that retrieves information from multiple PDF documents instead of a single document.

The chatbot searches across all uploaded PDFs and combines the relevant information to answer the user's question.

Simple Definition

Instead of searching one book, the AI searches an entire digital library.

Example

Uploaded PDFs:

- Employee Handbook.pdf
- Company Policy.pdf
- Leave Policy.pdf
- Internship Guide.pdf
- HR Rules.pdf

User asks:

"How many casual leaves are allowed?"

The chatbot searches all PDFs and returns the most relevant answer.

Part 3: Why Do We Need Multi-PDF RAG?

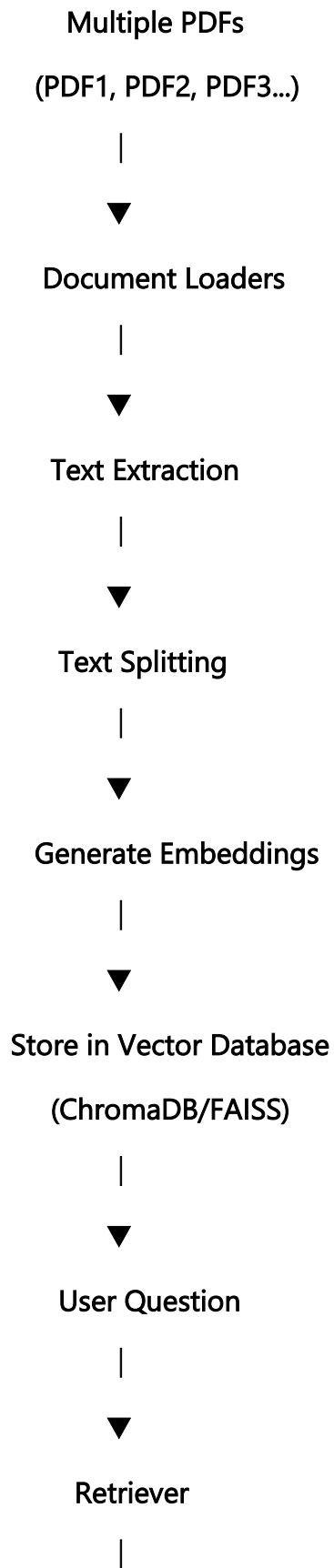
In companies, information is rarely stored in one document.

Examples:

- HR policies
- Employee handbook
- Product manuals
- Legal contracts
- Research papers
- Technical documentation

A Multi-PDF RAG system allows the AI to search all of them together.

📌 Part 4: Multi-PDF RAG Workflow



▼
Retrieve Relevant Chunks

|

▼
Prompt Template

|

▼
LLM

|

▼
Final AI Response

📌 Part 5: Loading Multiple PDFs

Instead of loading one PDF, we load all PDF files from a folder.

Example Folder

Documents/

|—— HR.pdf

|—— Internship.pdf

|—— CompanyPolicy.pdf

|—— LeavePolicy.pdf

|—— Handbook.pdf

Python Example

```
import os
```

```
from langchain_community.document_loaders import PyPDFLoader
```

```
documents = []
```

```
folder = "Documents"
```

```
for file in os.listdir(folder):
```

```
    if file.endswith(".pdf"):
```

```
        loader = PyPDFLoader(os.path.join(folder, file))
```

```
        documents.extend(loader.load())
```

```
print(f"Loaded {len(documents)} pages.")
```

Part 6: Splitting Multiple Documents

Large PDFs must be divided into smaller chunks.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
splitter = RecursiveCharacterTextSplitter(
```

```
    chunk_size=500,
```

```
    chunk_overlap=100
```

```
)
```

```
chunks = splitter.split_documents(documents)
```

```
print(len(chunks))
```

Part 7: Creating Embeddings

Each chunk is converted into a vector.

```
from langchain_huggingface import HuggingFaceEmbeddings
```

```
embedding = HuggingFaceEmbeddings(  
    model_name="sentence-transformers/all-MiniLM-L6-v2"  
)
```

Each chunk now has its own embedding.

📌 Part 8: Store Everything in ChromaDB

```
from langchain_chroma import Chroma
```

```
vector_db = Chroma.from_documents(  
    documents=chunks,  
    embedding=embedding,  
    persist_directory="./chroma_db"  
)
```

Now all document chunks from every PDF are stored in one vector database.

📌 Part 9: Searching Across Multiple PDFs

User Question:

"What is the internship duration?"

Retriever searches:

- Internship.pdf
- Handbook.pdf
- CompanyPolicy.pdf

It returns only the most relevant chunks.

✦ Part 10: Source Documents

A good RAG chatbot should show where the answer came from.

Example:

Answer:

Internship duration is 3 months.

Source:

Internship.pdf

Page 5

This increases user trust.

✦ Part 11: Handling Duplicate Information

Sometimes two PDFs contain the same information.

Example:

HR.pdf

Office starts at 9 AM.

Handbook.pdf

Office timing is 9 AM.

The retriever may return both chunks.

The LLM combines them into one clear answer instead of repeating the same information.

📌 Part 12: Challenges in Multi-PDF RAG

- Duplicate information
- Conflicting information between documents
- Large storage requirements
- Slower retrieval with many documents
- Keeping the vector database updated

📌 Part 13: Best Practices

- ✓ Use meaningful file names.
- ✓ Store metadata (file name, page number).
- ✓ Choose a suitable chunk size.
- ✓ Remove duplicate chunks.
- ✓ Rebuild embeddings when documents change.

📌 Part 14: Real-World Applications

Company Knowledge Assistant

Search HR, finance, and IT policies.

Legal Assistant

Search contracts, agreements, and regulations.

Medical AI

Search medical books and treatment guidelines.

University Assistant

Search lecture notes, textbooks, and assignments.

Research Assistant

Search multiple research papers simultaneously.

📌 Part 15: Advantages

- ✅ Search multiple documents at once.
 - ✅ More complete answers.
 - ✅ Saves time.
 - ✅ Supports enterprise knowledge management.
 - ✅ Easily scalable.
-

📌 Part 16: Limitations

- ❌ More storage needed.
 - ❌ Indexing takes longer.
 - ❌ Retrieval becomes slower as documents increase.
 - ❌ Duplicate information must be handled carefully.
-

📌 Part 17: Theory Questions

1. What is a Multi-PDF RAG system?

A Multi-PDF RAG system retrieves information from multiple PDF documents and uses it to answer user questions.

2. Why is Multi-PDF RAG important?

Most organizations store information across many documents. Multi-PDF RAG allows AI to search all of them together.

3. What is the role of metadata?

Metadata stores additional information such as the file name, page number, author, or document type. It helps identify the source of retrieved information.

4. Why do we split documents into chunks?

To make retrieval faster and allow the LLM to process manageable amounts of text.

5. How does a retriever work with multiple PDFs?

The retriever searches embeddings from all uploaded documents and returns the most relevant chunks regardless of which PDF they belong to.

Part 18: Interview Questions

Explain the workflow of a Multi-PDF RAG system.

The system loads multiple PDFs, extracts text, splits it into chunks, generates embeddings, stores them in a vector database, retrieves relevant chunks based on the user's question, and sends those chunks to an LLM to generate the final answer.

Why is metadata important in RAG?

Metadata identifies the source document and page number, making responses more trustworthy and easier to verify.

How can duplicate information be handled?

By removing duplicate chunks during preprocessing or combining similar retrieved chunks before generating the response.

What challenges arise when working with many PDFs?

Larger storage requirements, slower indexing, slower retrieval, duplicate information, and conflicting information between documents.

Why is Multi-PDF RAG useful for enterprises?

Because enterprise knowledge is distributed across many documents, and Multi-PDF RAG enables employees to search all of them through a single AI assistant.

Tasks for Day 76

Theory

1. Define Multi-PDF RAG.
 2. Explain the workflow of a Multi-PDF RAG system.
 3. Why is metadata important?
 4. What are the challenges of Multi-PDF RAG?
 5. Explain how duplicate information is handled.
-

Practical Task 1

Create a folder containing at least five PDF documents.

Practical Task 2

Write a program to load all PDFs from the folder.

Practical Task 3

Split all documents into chunks and generate embeddings.

Practical Task 4

Store the chunks in ChromaDB and create a retriever.

Practical Task 5

Ask questions that require information from different PDFs and verify that the chatbot retrieves the correct sources.